Pattern (what deviates from randomness, makers of some object/event) Discrimination: (1) feature (measurements or attributes that represent an object process or event that enable class assignment. They should be discriminative and not redundant) extraction, mapping between raw data space and the feature space; **(2) classification,** mapping between feature space and interpretation space. We do classification based on similarity (distance between feature vectors. **Stages of RP system** include measurement/sensing, raw data preprocessing (optional), feature extraction, feature preprocessing (optional), feature selection/reduction (optional), classification; postprocessing; performance evaluation (testing.) **Main concerns:** difficult to known if we have a sufficiently large and representative dataset; features should be discriminative and not redundant, there should be an adequate parameter search for any method chosen; evaluate model with different feature/parameter sets, for robustness; consider complexity vs performance trade-off. **Precision: TP/TP+FP**, **Acc = TP+TF/all, Recall/Sensitivity = TP/TP+FN, Specificity = TF/TF+FP, F1 = 2*Prec*Recall/Prec+Recall D0=w0/||w|| D=|d(x)|/||w||** a problem with dimension D and polinomial degree=0 has **D'=(O+D)!/(O!D!)** parameters. **Decision functions:** maps between feature space and interpretation space, splits IS into decision regions, if we apply a monotonic function to it we get exact same classification; also decision function can be a linear combination of transformations to features. This is helpful to turn complex non-linear problems into linearly separable problems in higher dimension or transformed spaces. Polynomial d(x) requires setting **(k+d)! / (k! d!)** parameters, with d=n_features and k=max_pol_degree.

Multi-class classification: divide into binary problems. Can be **absolute separation** (one-vs-all, we develop C classifiers, classifier i tells us whether sample belongs to i or not, and then we make hierarchical decisions) or **pairwise separation** (one-vs-one, we develop (c! / (2!(n-2)!) classifiers, each tells us what class in that pair the point most likely belongs to, final decision through majority voting. **Covariance Matrix:** indicates how variables vary and co-vary. It's symmetric. It's related to the **Mahalanobis distance:** adjusts to many cluster shapes, ie, data distribution, it's *scale invariant* (e. circle -> ellipsis). The mahalanobis decision surface between class 1 and 2 is linear if C1=C2 and quadratic otherwise. **PCA:** find a reduced set of new variables with less redundancy, minimizing loss. Project data into directions (eigenvectors of C) that maximize variance. The eigenvalues represent the variance along those directions. **Steps:** standardize data [(f-m)/s], compute C; find eigenvalues y such that **| y I - C | = 0** and then eigenvector matrix such that **(yI-C)=0.** Project data according to important PCs. **Kaiser Criterion:** keep eigenvectors with eigenvalues > 1; **Scree test:** discard eigenvalues after plot stabilizes (subjective.). PCA is **unsupervised** (doesn't need labels) which can be good but also means we can cut features because they don't contribute to variance but they actually contribute to overall discrimination. Also PCs are linear transformations, so they struggle with non-linear relationships. PCs have no physical/semantic meaning. **Visualization:** graphical inspection of features helps see their discriminative power. Histograms let us know where data is located (mean/median), how spread it is (std/quartiles), whether it's symmetric or skewed and whether there are outliers. We should do a distribution model assessment to know wether we can apply parametric tests to it. **Kolmogorov-Smirnov (K-S):** quantifies the difference between the empirical cumulative distribution function (CDF) of the data and the theoretical CDF of a normal standard distribution. The null hypothesis states that the data is drawn from a normal distribution. **Shapiro-Wilk** is the same but for n<25. **Techniques to avoid overfitting:** Cross-validation, regularization to constrain complexity, pruning (in decision trees, ensemble methods (bagging/random forest) **Kruskal-Wallis:** non-parametric statistical test to assess features discriminative power. Formula: **(1)** sort feature value (separate each class)s; **(2)** assign ordinal rank; **(3)** compute the H statistics: as high as H is, the more discriminative the feature. **H0:** samples from different classes are drawn from the same population (no discriminative power.) **Feature Redundancy:** use correlation matrix. **Dimensionality Ratio problem:** dr = n / d, where n is the number of samples and d number of dimensions. If we have low n, we have poor representation of problem, however, with too many dimensions we can overfit data. This is the **curse of dimensionality:** there exists a critical feature dimension (CFD) above which performance degrades (we can't just add more dimensions while keeping the number of training samples the same). In geometric pov, if we have d features each divided in m buckets, then we have m**d hypercubes dividing space – grows exponentially w d and can lead to very sparse space, representing the problem poorly. CFD not theoretically ascertainableThe increase of feature space can cause overfitting. **MDC:** consider class means as class prototypes (representatives) and assign to each new sample the class of the nearest prototype. **Euclidean MDC:** the decision function forms hyperplanes perpendicular to the segment linking the means and passing at half-distance. **For Mahalanobis MDC:** we can use as C the pooled covariance matrix (mean of Cs of classes.) This MDC equals the Euclidean MDC when C is proportional to I. **Fisher LDA:** goal is to separate samples of diff classes by placing them in a space maximizing between-class separability and minimizing within-class variability. We can use this for dimensionality reduction and linear classifier (**Fisher LDA**). LDA assumes implicitly that the C of each class is equal, because the same within-class scatter matrix is used for all the classes considered. We find the optimal projection vector, w and project data according to that direction, then use euclidean MDC. LDA reduces data to at most C-1 dimensions, and it has trouble with some layouts, for example when centroids of classes are the same (ex. Concentrical) or when there's a big overlap between classes in any direction. **Advantages vs Disadvantages LDA:** advantages: computationally efficient, noise reduction, enhance classifier performance by maximizing separability, can be easily interpretable. Disadvantages: assumes data has the same covariance matrix, works best when linearly separable, difficult with some layouts, can produce at most C-1 d. S_B*W = lambda*S_W*W

- The H statistic is:
  - $H = \frac{12}{n(n+1)}\sum_{i=1}^{k} n_i(\bar{R}_i - \bar{R})^2$
  - $\bar{R}_i$ → Average of ranks for the samples belonging to **class i**
  - $\bar{R}$ → Average of all ranks for all classes
  - n_i → Number of samples belonging to **class i**
  - n →Total number of samples

- Decide ω_1 if $\frac{p(\mathbf{x}|\omega_1)}{p(\mathbf{x}|\omega_2)} > \frac{\lambda_{12}P(\omega_2)}{\lambda_{21}P(\omega_1)}$ **Bayesian Classifier:** takes a probabilistic approach, makes decisions based on probabilities and their associated costs. **Training** involves getting a priori probabilities and class conditional probabilities. **A priori probability P(wi):** probability of class wi without knowing any specific measurements or data distribution. **Class conditional probability density function p(x | wi):** likelihood of measurement x to occur when the class is wi, we can estimate this using training data. **A posteriori probability, P(wi | x):** probability of class wi when the measurement value is x. **Minimum Error Classification Starting Point:** we want to choose the most probable class, ie, choose w1 if P(w1 | x) > P(w2 | x). **Decision boundary:** we can say decision boundary is d(x) = P(x | w1)*P(w1) – P(x | w2)*P(w2), or d(x) = g1(x) – g2(x), with gi(x)=ln(P(x|wi)*P(wi)) = ln(P(x|wi) + ln(P(wi). This is helpful when we equal P(x|wi) to the normal distribution. If we develop the function d(x) = g1(x) – g2(x) using the formula to the left, and assume C1 and C2 are equal (like Mahalanobis MDC) then we achieve a linear decision surface with the formula basically as the same as Mahalanobis MDC but with an extra factor (ln of prevalence ratio). If the prevalence ratio is 1 (same prevelance) then ln(1)=0 and Bayesian behaves exactly like Mahalanobis. **If equal prevalence and likelihood:** the threshold is half distance from means, incorrect classification equal for both classes, identical to MDC. **If unequal prevalence and equal likelihood:** decision threshold displaced towards the class with least prevalence in order to reduce number of incorrect classifications for the higher prevalence class. **Minimum Risk Classification Starting Point:** decide w1 if l1 < l2. Li is the loss function for choosing I, aka, describes how much expected loss is associated with that decision. Se desenvolvermos **l1<l2** vai dar algo semelhante ao LRT. If **equal loss** then the right-hand side is simply the prevalence ratio, aka it would equal the **minimum error classification. Loss function:** λ(ai,wj) cost of taking action ai when the true state is wj **Risk/Total Risk:** expected loss for a specific decision given an input x (loss over all possible true states weighted by probabilities: R(ai|x) = sum{i}λ(ai,wj)P(wj|x) **Bayes Risk:** Minimum possible risk achievable for a given problem (depends on the data itself, not the classifier, obtained when bayes rule is perfectly followed). **Bayes Rule:** Select action a with lowest Total Risk R(ai | x). **Strategies 4 obtaining decision rule: Generative**: Estimate P(l|w) and P(w) then apply Bayes. **Discriminative**: Learn decision boundary or the posterior prob. directly w/o underlying distributions. **Generalization:** Good in unseen data, **Memorization:** Overfit. **Resubstitution:** whole set X used for training and testing. Corresponds to setting the error estimate on the training set (apparent error). This is an optimistic and low variance error estimate. **Holdout:** total dataset randomly or not divided into two disjoint sets, one for training and one for testing .Improve testing error estimate by doing different runs. **Cross-validation (K fold):** divide data into k>1 subsets of randomly selected patterns, keep one fold with n/k patterns for testing and the remaining (n-1) parts for training, rotate k times so each fold can be testing fold and average results. **Stratified cross-validation:** ensure class prevalence along folds to handle imbalanced data. **Bootstrap:** generate artificial data samples by randomly selecting existing samples with uniform distribution. **Three set validation:** we use training and valiation subsets to choose the best model (ex. Set of parameters) and then test that on a never-before-used testing subset.**Bias:** How far is the average performance from optimal. **Variance:** How much the performance(decision boundary) varies over different training sets **ROC:** plots the proportion of correct responses (TP or sensitivity) against the FP rate as a threshold changes. Represents trade-off between sensitivity and specificity. A perfect classifier would have AUC=1. Random classifier would have a diagonal ROC (45º degree.).Best threshold(T) can be found with minimum distance to the optimal performance (FPR=0, SS=0) min_T[(1-SS(T))^2 + FPR(T)^2) or max_T(SS(T) + SP(T) - 1) **Parzen Window:** kernel density estimation. Creates histograms. **Non-parametric. KNN:** consider k points that are the nearest neighbors of x using a certain distance metric; the label of x depends on the class label that is found in majority among the k neighbors. It's **non-parametric** (doesn't assume about data distribution) and a **lazy** classifier because training only implies storing the feature vectors and class labels of training samples. This is good because it requires less computation for training. Also it's instance based therefore adapts locally to data structure (good for non linearly separable problems). A low k overfitts, a high k increases the error rate. **Filters:** pre-processing methods independent from classifier; simple features assume independence between features and only look at relevance (discriminative power.) Ideally it would be combined with a redundancy analysis (eg. minimum redundancy maximum relevancy algorithm (mRMR). **Advantages:** fast and computationally efficient, may provice a generic selection independent of classifier. **Disadvantages:** assume independence between features. Based just on data, not in classification performance. **Wrappers:** based on classifier performamnce: we train a model with different subsets of features and choose the one that yields best performance. **Advantages:** high accuracy, overfitting control associated with classifier. **Disadvantages:** computationally expensive and lacks generality. **Embedded:** use all features and define their relevance internally during learning. More important features affect the internal parameters of the classifier more. **VC dimension, h:** largest number of samples that can be correctly separated (shattered) by a classifier during training, considering all possible ways to label these samples. **Structural risk:** Structural risk refers to minimizing a bound on the true (generalization) error by balancing empirical risk with model complexity, selecting among nested function classes so as to control overfitting and achieve the best trade-off between fit to data and capacity.

SVM: define an optimal hyperplane that minimizes the training error and maximizes the separation margin among classes (minimizing structural risk.) It can extend this definition to non-linearly separable problems by having a penalty for misclassifications (soft-margin) It can also use kernel trick to map data to high dimensional space where it's easier to classify with linear decision surfaces. **MARGIN** = 2/||w|.| **Margin**, r. K is usually 1. Support vectors are those closest to decision boundary, aka **wt * x + b = +- k.** O SVM resolve o problema convexo que equivale a maximizar a margem entre as duas classes sob a restrição de classificar corretamente com margem pelo menos 1. Resolve-se com o Lagrange multipliers method: apenas support vectors ficam com lagrange multipliers não nulos. **SVM soft-margin:** this method handles non-linearly separable data by accounting the deviations from the ideal separable problem, it allows patterns to fall within the margin and in the opposite region but penalizes them. It adds a **free parameter, C,** that influences margin: **higher C means smaller margins**, aka hard penalization of wrong classifications, but can overfit. **Smaller C means larger margins**, soft penalization of the wrong classification. **Kernel Trick:** we can use functions to transform data to higher dimensional spaces to make problems linearly separable. But this can be computationally expensive to compute exhaustively. Kernel functions allow us to use SVMs without explicitly mapping the data; in the svm formulas we simply substitute inner products xiTxj for the kernel function K(xi, xj) = f(xi)^T * f(xj). **Gaussian kernel parameter:** this y parameter controls the locality of influence of support vectors: a large y makes each support vector affect only a small neighborhood (complex, wiggly boundary) whereas a small y yields broader influence and a smoother decision surface. **Multi-class SVM:** decompose problem in binary problems using one vs all or one vs one. We can use the **Error-Correcting Output Codes (ECOC) model**, which requires a **coding design matrix** to determine the classes that the binary classifiers will try to discriminate and set a base against which will decode the final design. To decode, the most straightforward approach is compute **hamming distance** between classifiers output and each line of the coding design matrix and attribute the class associated to the minimum distance. Hamming distance is count the number of different cells. **Get non-linear SVM w and w0 given support vectors without lagrange multipliers:** Identify SVs x1 and x2 in the original plane (on top of boundaries) -> map them to z1 and z2 with kernel -> bias = (z1_1 + z2_1)/2 -> weights vector according to the slope of the line in the middle of them. That will give us w1z1 + w2z2 + b, change to w1(x1^2)+w2(x2^2) + b if the kernel transformation is f(x) = (x1^2 x2^2)^T. **Hybrid Classifier:** combines two or more different classification methods to exploit their complementary strengths and improve overall performance.

Occam's Razor: when you have two competing theories which make exactly the same predictions, the simpler is the better. Aka, we should select the simplest model because it minimizes the number of incorrect assumptions. **No Free Lunch Theorem:** no algorithm is superior to any other over all possible classification problems. **Bias:** error rate due to the average performance of the classifier, how far is the average performance from optimal?. **Variance:** error rate due to training set changes, how does does it vary for different training sets? **Bagging:** draw n'<N samples with replacement from D several times, use each set to train a classifier (parallel), combine results of each classifier to the final classification usually by majority voting. This decreases variance (use case: random forests.) **Boosting:** learn a strong classifier by combining weak classifiers (classifiers with low bias and variance, only slightly better than random guessing.) We learn weak classifiers iteratively (not parallel) and reweigh data points. The final decision is a weighted sum of all the weak learners. (use case: AdaBoost -> formula to the left) **Stacking:** combine several heterogeneous models by training an additional classifier with the individual decision function values. Learning from diverse predictions may lead to improved overall classification. **Generalized Probabilistic Classifier Combination:** combine results of R classifiers where each classifier k outputs the probabilities of each class Pk(Wj | x.) We can apply the **Product Rule (product of every likelihood, sensitive to very low values)** or the **sum rule (sum of every likelihood),** sum rule found to perform well in practice. If prod(k=1->K)(P(wj)|x) > prod(k=1->K)(P(wi)|x) assign wj, else assign wi.\ **Clustering. Dendogram:** Tree diagram frequently used to illustrate the arrangement of clusters produced by hierarchical clustering. Always a binary tree. Similarity is the height of the lowest internal node shared by both**Types:** *Hierarchical/Tree:* Use linkage rules to create hierarchical sequence of clustering solutions. *Centroid adjustment algorithms / partitioning*: Based on an initial proposal for the prototypes (centroids), the algo iteratively evaluates and adjusts centroid position. *Density-based:* Join areas of high pattern density into clusters. Good for arbitrarily-shaped clusters because it can be formed as long as dense areas can be connected. *Distribution-based:* Estimate parametric data distributions from data, e.g., gaussian. *Others!* **Hierarchical Clustering:** *Agglomerative:* Start w/ *N* clusters where each cluster is composed of a single sample -> Merge two nearest clusters -> repeat until all clusters are part of a single cluster -> output dendrogram (hier. tree that shows how each cluster unites). *Divisive*: Start with all in 1 cluster -> split in two -> … **Linkage rules:** What will be the new distance between the join clusters and other clusters? *Single:* Dissimilarity given by the closest elements (good for clusters with filamentary shape. promote connectedness). *Complete:* Dissimilarity given by the furthest elements (good for compact clusters. original spherical-shaped clusters). *Average:* Pair-group using arithmetic average (effective for several types of clusters). *Centroid:* Euclidean distance between centroids of the two clusters (effective for several types of clusters). *Ward's method:* Sum of the squared within-cluster distances (minimize intra-cluster variability and consequently maximize cluster separability). *Lance-Williams formula:* update the distance matrix in agglomerative methods after merging clusters (the d in the formula can be the distance for any of the previously mentioned methods). **Centroid adjustment algorithms (K-MEANS):** Assign data point to clusters -> move centroid describing the clusters. Number of clusters need to be defined a-priori (we can use hierarchical clustering in a small portion of data for initialisation. Minimize and overall within cluster distance from the patterns to the centroids (loss function to the right). K-means works if: Clusters are spherical -> clusters are well separated -> clusters are of similar volumes -> clusters have similar number of points ***K-means Alg***: assign each datapoint to the cluster represented by the nearest centroid -> compute, for the previous partition, the new centroids -> repeat until max num of iteration exceeded or there are no more changes on the centroids. is robust to outliers points in very low populated area. Do not need an initial guess for the number of clusters. Deal with arbitrary cluster shapes **Density Algorithms (DBSCAN):** Clusters are dense areas separated by areas of lower density. Defined by two parameters: ε (radius of neighborhood with respect to some point) minPts (min number of points required to consider a region dense. *Advantages:* robust to outliers (points in very low populated areas); do not need an initial guess for the number of clusters; deal with arbitrary cluster shapes. *Point Types:* Core Point: If at least minPts points are withing a distance ε of it (including the point itself). Border Point: a point that has at least one core point at a distance ε and if the number of points inside the volume defined by ε is less than minPoint. Noise Point: Not core/border. *Reachability:* If a data point can be accessed from another datapoint (directly or indirectly). *Connectivity:* If two datapoints belong to the same cluster or not. *Two points in DBSCAN can be:* Directly density-reachable if d(y,x) < ε and y is core point, density-reachable if they can be reached indirectly, density connected if both x and y are density-reachable from O. ***DBSCAN Alg.:*** Pick a point not assigned to a cluster or classified as outlier -> compute its neighborhood to determine if it's a core point -> if core point: start cluster around this point, assign label to point and all neighbours, implement neighborhood jumps to find all density-reachable points and add them to the cluster, if outlier is added, change status to border point; if not core point: label point as outlier -> repeat until all points are labeled as cluster or outlier. ***Setting hyperparameters:*** *minPts:* minPts >= numdimensions+1. This prevents the alg. from forming clusters that do not have enough density relative to num of dimensions. If we set minPts to <=2, The algorithm effectively becomes single-link agglomerative hierarchical clustering (any two points within distance ε are connected, cutting the dendrogram at height ε rather than filling dense regions). ε: When MinPts >= 3, develop a k-distance graph -> define k = minPts-1. For every data point in the set, calculate distance to the k-th nearest neighbor. Sort these calculated distances and plot them. the elbow is the optimal ε (threshold where points stop behaving like noise and start being part of a cluster). **Cluster evaluation:** *Unsupervised/internal:* No labels, and in general, evaluate if clusters are compact (small within cluster distances) and if clusters are well separated (large between clusters distances) (e.g. Davies-Bouldin idx, Silhouette, Dunn Index, …). *Supervised/external:* Need labels to evaluate. (e.g. Purity / Accuracy, Precision, Recall, …). **Performance Evaluation.** Confusion Matrix with TP, TN, FP, FN. Accuracy = (TP+TN)/(TP+TN+FP+FN). Sensitivity/Recall = TP/(TP+FN). Specificity = TN/(TN+FP). Precision = TP/(TP+FP). F1-score = 2·(Precision·Recall)/(Precision+ Recall). Macro averaging computes metrics per class and averages them (class- -balanced); Micro averaging computes metrics globally and is dominated by majority classes (≈ accuracy). For class imbalance: Binary → use Recall, Specificity, F1 or Balanced Accuracy; Multiclass → use BACC/BACCw. **Decision Trees.** Classification through recursive if-then rules; each root-to-leaf path is a conjunction, the full tree is a disjunction. Advantages: interpretable, fast, handles mixed data. Disadvantages: greedy search, high variance, sensitive to data changes. **Entropy and Information Gain (ID3).** Entropy measures class heterogeneity: H(S)=−∑pᵢlog₂pᵢ. Information Gain: Gain(S,A)=H(S)−∑ᵥ(|Sᵥ|/|S|)H(Sᵥ) S=dataset Sv subset of the dataset using feature A value v the second term of the subtraction are the expected values of entropy after partitioning S using feature A value v. ID3 selects the feature with highest IG recursively until entropy is zero. IG is biased toward features with many values and sensitive to noise and class imbalance. **CART(Non-parametric).** Uses Gini impurity (low Gini impurity instances belong to a single class, High gini impurity means a more mixed class distribution. G=1−∑pᵢ², binary splits only, supports classification and regression. Overfitting controlled by pruning, depth limits, minimum samples per leaf, cross-validation or Random Forests the reduction is the Gini impurity after Reduction = Gini impurity for the original dataset - Gini impurity after splitting by each feature.**Naïve Bayes.** Assumes conditional independence between features given the class. Likelihood Ratio Test: p(x|ω₁)/p(x|ω₂) > (ω₂)/P(ω₁) ⇒ choose ω₁. Capital P denotes discrete probabilities, lowercase p denotes probability density functions; p(x) is the evidence.Advantages: Versatility: handles both classification and regression tasks. Interpretability: The outcome of the CART algorithm are visually intuitive and easy to understand. Non parametric: does not make any underlying assumptions about the distribution of the data making it adaptable to diverse datasets. Handles mixed data types: The algorithm can easily manage datasets containing both categorical and numerical features. Automatic feature selection: Inherent in its design, CART will naturally give importance to the most informative and numerical features. Limitations: overfitting, sensitive to data changes binarity splits, local optima (its greedy), difficulty with non linearly separable data. Pruning, depth limit, cross validation and random forests can solve overfitting. **Bias–Variance Decomposition.** Total error = bias + variance. High bias → underfitting; high variance → overfitting. Overfitting corresponds to very low training error and high testing error, often due to high model complexity (high VC dimension).**Feature Selection Additions.** ROC/AUC can be used to assess individual feature relevance; higher AUC implies better discriminative power.

Likelihood ratio test (LRT)

Likelihood ratio Prevalence Threshold

$$\mathbf{w} = \mathbf{C}^{-1}(\boldsymbol{\mu}_i - \boldsymbol{\mu}_j);\ w_0 = -\frac{1}{2}(\boldsymbol{\mu}_i^T\mathbf{C}^{-1}\boldsymbol{\mu}_i - \boldsymbol{\mu}_j^T\mathbf{C}^{-1}\boldsymbol{\mu}_j) + \ln\left(\frac{P(\omega_i)}{P(\omega_j)}\right)$$

$$\hat{p}(\mathbf{x}) = \frac{1}{N}\frac{1}{V}\sum_{i=1}^{n}\varphi\left(\frac{\mathbf{x}-\mathbf{x}_i}{h}\right)$$

- $R(\%) = \frac{\sum_{i=1}^{p}\lambda_i^2}{\sum_{i=1}^{M}\lambda_i^2}$x100

Univariate normal pdf:

$$N(\mu,\sigma^2) \approx p(x) = \frac{1}{\sqrt{2\pi}\sigma}e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

$$\hat{p}(\mathbf{x}) = \frac{1}{NV}\sum_{i=1}^{N}\frac{1}{(2\pi)^{D/2}}e^{-\frac{1}{2h^2}(\mathbf{x}-\mathbf{x}_i)^T(\mathbf{x}-\mathbf{x}_i)}$$

- $c = 2$ (two class problem):
  - $R = \lambda_{12}\int_{\mathcal{R}_1} p(\mathbf{x}|\omega_2)P(\omega_2)d\mathbf{x} + \lambda_{21}\int_{\mathcal{R}_2} p(\mathbf{x}|\omega_1)P(\omega_1)d\mathbf{x}$

$$\mathbf{w} = \mathbf{S}_w^{-1}(\mathbf{m}_1 - \mathbf{m}_2) \quad r = \frac{2k}{||\mathbf{w}||}$$

$$d(x) = \sum_{i=1}^{N_{SV}}\alpha_i y_i K(x_i, x) + b$$

$$K(\mathbf{x}_i, \mathbf{x}) = e^{-\frac{(\mathbf{x}-\mathbf{x}_i)^2}{2\sigma^2}} = e^{-\gamma(\mathbf{x}-\mathbf{x}_i)^2}$$

$$\text{Maximize } L_D(\alpha) = \sum_{i=1}^{N}\alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y_i y_j K(x_i, x_j)$$

Final classifier: $H(\mathbf{x}) = \text{sign}\left(\sum_{j=1}^{M}\alpha_j H_j(\mathbf{x})\right)$. ❖$K(\mathbf{x}_i, \mathbf{x}) = f(\mathbf{x}_i)^T f(\mathbf{x})$

❖$\tilde{\mathbf{w}} = \sum_{i=1}^{n}\tilde{\alpha}_i y_i \mathbf{x}_i$,

❖$\tilde{b} = -\frac{1}{2}\tilde{\mathbf{w}}^T(\mathbf{x}_p + \mathbf{x}_m)$. With $\mathbf{x}_p$ and $\mathbf{x}_m$ being **any support vector** from +1 and -1 classes, respectively.

❖$g_k(\mathbf{x}) = \mathbf{m}_k{}^T\mathbf{C}^{-1}\mathbf{x} - 0.5\mathbf{m}_k{}^T\mathbf{C}^{-1}\mathbf{m}_k$ $\quad d_{i,j}(\mathbf{x}) = (\mathbf{m}_i - \mathbf{m}_j)^T\mathbf{C}^{-1}[\mathbf{x} - 0.5(\mathbf{m}_i + \mathbf{m}_j)]$

- $\mathbf{w} = \mathbf{S}_w^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$; $\mathbf{S}_W = \mathbf{S}_1 + \mathbf{S}_2 = (N_1 - 1)\,\mathbf{C}_1 + (N_2 - 1)\,\mathbf{C}_2 = 2(N - 1)\,\mathbf{C}$

$$\mathbf{S}_w^{-1} = \frac{1}{2(N-1)}\mathbf{C}^{-1} \text{ thus } \mathbf{w} = \frac{1}{2(N-1)}\mathbf{C}^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$$

$$d(\mathbf{x}) = \mathbf{w}^T\mathbf{x} - \frac{\mathbf{w}^T\mathbf{m}_1 + \mathbf{w}^T\mathbf{m}_2}{2}$$

Single Link	$a_i = a_j = 0.5;\ b = 0;\ c = -0.5$
Complete Link	$a_i = a_j = 0.5;\ b = 0;\ c = 0.5$
Average Link	$a_i = \frac{n_i}{n_i+n_j};\ a_j = \frac{n_j}{n_i+n_j};\ b = c = 0$
Centroid	$a_i = \frac{n_i}{n_i+n_j};\ a_j = \frac{n_j}{n_i+n_j};\ b = -\frac{n_i n_j}{(n_i+n_j)^2};\ c = 0$
Ward's Method	$a_i = \frac{n_i+n_k}{n_i+n_j+n_k};\ a_j = \frac{n_j+n_k}{n_i+n_j+n_k};\ b = -\frac{n_k}{n_i+n_j+n_k};\ c = 0$

❖$d(\omega_{i+j}, \omega_k) = a_i d(\omega_i, \omega_k) + a_j d(\omega_j, \omega_k) + bd(\omega_i, \omega_j) + c|d(\omega_i, \omega_k) - d(\omega_j, \omega_k)|$

Regularization as embedded selection: LASSO (L1) forces irrelevant feature weights to zero; Ridge (L2) shrinks weights to reduce variance.**Distance Metrics.** A valid metric satisfies non-negativity, identity, symmetry and triangle inequality. A norm also satisfies d(ax,ay)=|a|d(x,y). **Parametric (naive bayes, LDA...):** A model is assumed for the likelihoods (fixed parameters, low flexibility and fast training speed) **Non-parametric learning:** Attemp to estimate the density directly from the data without assuming a particular form for the distribution (grows with data, high flexibility, slower training speed KNN, Trees, SVM(kernel)) **Use parametric classifiers if dataset is small, need fast inference and interpretability is important.**

```python
def roc_auc_analysis(X, y, feature_names):
    X = np.asarray(X)
    y = np.asarray(y)
    pl = np.unique(y)[1]
    feature_names = np.array(feature_names)
    roc_auc = np.zeros(X.shape[1])
    for i in range(X.shape[1]):
        fpr, tpr, _ = roc_curve(y, X[:, i], pos_label=pl)
        roc_auc[i] = auc(fpr, tpr)
    roc_auc = np.maximum(roc_auc, 1 - roc_auc)
    sortIx = np.flip(np.argsort(roc_auc))
    print("Sorting according to ROC-AUC:")
    for i in sortIx:
        print(feature_names[i] + "-->" + str(roc_auc[i]))
    roc_auc_sorted = roc_auc[sortIx]
    fnames_sorted = feature_names[sortIx]
    return roc_auc_sorted, fnames_sorted
```

```python
def correlation_matrix(X, Hs, feature_names):
    X = np.asarray(X)
    top_feature_names = [h[0] for h in Hs[:9]]
    feature_indices = []
    for fname in top_feature_names:
        try:
            idx = feature_names.index(fname)
            feature_indices.append(idx)
        except ValueError: pass
    X_top = X[:, feature_indices]
    corrMat = np.corrcoef(X_top.T)
    return corrMat
```

```python
def test_random_forest(model, X, y):
    X = np.asarray(X)
    y_true = np.asarray(y)
    clf = model["clf"]
    X_test = X
    y_pred = clf.predict(X_test)
    dec_scores = clf.predict_proba(X_test)[:, 1]
    TP = np.sum((y_true == 1) & (y_pred == 1))
    TN = np.sum((y_true == 0) & (y_pred == 0))
    FP = np.sum((y_true == 0) & (y_pred == 1))
    FN = np.sum((y_true == 1) & (y_pred == 0))
    ss, sp = TP / (TP + FN), TN / (TN + FP)
    acc = (TP + TN) / (TP + TN + FP + FN)
    return ss, sp, acc, f1_score(y_true, y_pred)
```

```python
def train_euclidean_distance(X, y, f_names=None):
    X, y = np.asarray(X), np.asarray(y)
    # unique_labels check in bayesian
    # f_names check in bayesian
    ix0 = np.where(y == 0)[0]
    ix1 = np.where(y == 1)[0]
    X_0, X_1 = X[ix0], X[ix1]
    mu_0 = np.mean(X_0, axis=0)
    mu_1 = np.mean(X_1, axis=0)
    model = {"feature_names": f_names,
    "mu_0": mu_0,"mu_1": mu_1}
    X_train, y_train = X, y
    dist_0 = np.linalg.norm(X_train - mu_0, axis=1)
    dist_1 = np.linalg.norm(X_train - mu_1, axis=1)
    y_pred_tr = (dist_1 < dist_0).astype(int)
    return model # , ...
```

```python
def kruskal_wallis_test(X, y, f_names=None):
    X, y = np.asarray(X), np.asarray(y)
    # unique_labels check in bayesian
    # f_names check in bayesian
    ix0 = np.where(y == 0)[0]
    ix1 = np.where(y == 1)[0]
    H_rank = []
    for i, fname in enumerate(f_names):
        H, p = stats.kruskal(X[ix0, i], X[ix1, i])
        H_rank.append((fname, H))
    Hs = sorted(H_rank, key=lambda x: x[1], reverse=True)
    return Hs
```

```python
def lda_analysis(X, y):
    X = np.asarray(X)
    y = np.asarray(y)
    lda = LinearDiscriminantAnalysis()
    lda.fit(X,y)
    transformed = lda.transform(X)
    return transformed
def lda_test(X, lda):
    X = np.asarray(X)
    transformed = lda.transform(X)
    return transformed[:, 0] # LD1
```

```python
def train_random_forest(X, y, f_names,
    n_estimators=100, max_depth=None):
    X = np.asarray(X)
    y = np.asarray(y)
    # unique_labels check in bayesian
    # f_names check in bayesian
    X_train, y_train = X, y
    clf = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        random_state=42,
        class_weight='balanced'
    )
    clf.fit(X_train, y_train)
    y_pred_tr = clf.predict(X_train)
    dec_scores_tr = clf.predict_proba(
        X_train)[:, 1]
    model = {
        "feature_names": f_names,
        "clf": clf,
        "n_estimators": n_estimators,
        "max_depth": max_depth
    }
    return model
```

```python
test_euclidean_distance(model, X, y):
X, y_true = np.asarray(X), np.asarray(y)
mu_0 = model["mu_0"]
mu_1 = model["mu_1"]
X_test = X
dist_0 = np.zeros(len(X_test))
dist_1 = np.zeros(len(X_test))
y_pred = np.zeros(len(X_test))
for i, sample in enumerate(X_test):
    dist_0[i] = np.linalg.norm(sample - mu_0)
    dist_1[i] = np.linalg.norm(sample - mu_1)
    y_pred[i] = 0 if dist_0[i] < dist_1[i] else 1
decision_scores = dist_0 - dist_1
```

```python
test_decision_tree(model, X, y):
X, y_true = np.asarray(X), np.asarray(y)
clf = model["clf"]
X_test = X
y_pred = clf.predict(X_test)
decision_scores = clf.predict_proba(X_test)
                                    [:, 1]
```

```python
def train_adaboost_custom(X, y, f_names,
            n_mod=5, learning_rate=1.0):
    X, y = np.asarray(X), np.asarray(y)
    # unique_labels check in bayesian
    # f_names check in bayesian
    X_train, y_train_01 = X, y
    y_train_pm1 = np.where(y_train_01 == 1, 1, -1)
    trained_models = trainAdaboost(X_train,
        y_train_pm1, n_mod, learning_rate)
    model = {"feature_names": f_names,
        "models": trained_models, "n_mod": n_mod}
    nMod = trained_models.shape[0]
    nsamp = X_train.shape[0]
    predTot = np.zeros((nMod, nsamp))
    for m in range(nMod):
        label = trained_models[m].predict(X_train)
        predTot[m, :] = trained_models[m].alpha * label
    decision_scores_tr = np.sum(predTot, axis=0)
    y_pred_pm1 = np.sign(decision_scores_tr)
    y_pred_tr = np.where(y_pred_pm1 == 1, 1, 0)
    y_train = y_train_01
    ...
    return model, ...
```

```python
def train_bayesian_classifier(X, y,
    feature_names=None, method_name="Unknown"):
    X = np.asarray(X)
    y = np.asarray(y)
    unique_labels = np.unique(y)
    if len(unique_labels) != 2:
        raise ValueError("Expected binary classification, "
                         f"got {len(unique_labels)} classes")
    if not np.array_equal(np.sort(unique_labels), [0, 1]):
        label_map = {unique_labels[0]: 0, unique_labels[1]: 1}
        y = np.array([label_map[label] for label in y])
    if feature_names is None:
        feature_names = [f"f_{i}" for i in range(X.shape[1])]
    X_train, y_train = X, y
    ix0 = np.where(y_train == 0)[0]
    ix1 = np.where(y_train == 1)[0]
    Pw0 = ix0.shape[0] / (ix0.shape[0] + ix1.shape[0])
    Pw1 = ix1.shape[0] / (ix0.shape[0] + ix1.shape[0])
    clf1 = mixture.GaussianMixture(n_components=1)
    clf2 = mixture.GaussianMixture(n_components=1)
    mod1=clf1.fit(X_train[ix0,:])
    mod2=clf2.fit(X_train[ix1,:])
    mean1 = mod1.means_.squeeze()
    mean2 = mod2.means_.squeeze()
    cov1 = mod1.covariances_[0]
    cov2 = mod2.covariances_[0]
    reg = 1e-6  # regularização p/ evitar matrizes singulares
    if cov1.ndim == 0:
        cov1 = np.array([[cov1 + reg]])
        cov2 = np.array([[cov2 + reg]])
    else:
        cov1 = cov1 + reg * np.eye(cov1.shape[0])
        cov2 = cov2 + reg * np.eye(cov2.shape[0])
    model = {
        "feature_names": feature_names,
        "mean0": mean1, "mean1": mean2,
        "cov0": cov1,  "cov1": cov2,
        "Pw0": Pw0, "Pw1": Pw1
    }
    logp0_tr = multivariate_normal.logpdf(
        X_train, mean=mean1, cov=cov1) + np.log(Pw0)
    logp1_tr = multivariate_normal.logpdf(
        X_train, mean=mean2, cov=cov2) + np.log(Pw1)
    y_pred_tr = np.where(logp1_tr >= logp0_tr, 1, 0)
    decision_scores_tr = logp1_tr - logp0_tr
```

```python
def train_mahalanobis_distance(f_names, data, ixHealthy, ixCancer):
    X_healthy = data.iloc[ixHealthy[0], :][f_names].to_numpy()
    X_cancer  = data.iloc[ixCancer[0], :][f_names].to_numpy()
    mu_healthy = np.mean(X_healthy, axis=0)
    mu_cancer  = np.mean(X_cancer, axis=0)
    if X_healthy.ndim == 1 or X_healthy.shape[1] == 1:
        var_healthy = np.var(X_healthy, ddof=1)
        pooled_var = (np.var(X_healthy, ddof=1) +
                      np.var(X_cancer, ddof=1)) / 2
        inv_cov = 1.0 / pooled_var
    else:
        cov_healthy = np.cov(X_healthy, rowvar=False)
        cov_cancer  = np.cov(X_cancer, rowvar=False)
        cov_pooled  = (cov_healthy + cov_cancer) / 2
        inv_cov = np.linalg.inv(cov_pooled)
    model = {"feature_names": f_names, "mu_healthy": mu_healthy,
        "mu_cancer": mu_cancer, "inv_cov": inv_cov}
    X_train = np.concatenate([X_healthy, X_cancer], axis=0)
    y_train = np.concatenate([np.zeros(len(X_healthy)),
                              np.ones(len(X_cancer))])
    if X_train.shape[1] == 1:
        muH = mu_healthy.flatten()
        muC = mu_cancer.flatten()
        inv_cov_s = inv_cov
        dx = ((X_train.flatten() - 0.5 *(muH + muC))
              * (muC - muH)) * inv_cov_s
    else:
        dx = ((mu_cancer - mu_healthy).T @ inv_cov @ (X_train.T
            - 0.5*(mu_cancer + mu_healthy)[:, np.newaxis])).flatten()
    y_pred_tr = (dx > 0).astype(int)
    ...
    return model #plus relevant metrics?
```

```python
def train_decision_tree(X, y, f_names=None,
            max_depth=5, criterion='gini'):
    X, y = np.asarray(X), np.asarray(y)
    # unique_labels check in bayesian
    # f_names check in bayesian
    X_train, y_train = X, y
    clf = DecisionTreeClassifier(max_depth=
            max_depth, criterion=criterion)
    clf.fit(X_train, y_train)
    y_pred_tr = clf.predict(X_train)
    ds_tr = clf.predict_proba(X_train)[:, 1]
    model = {"feature_names": f_names,
        "clf": clf, "max_depth": max_depth}
    return model
```

```python
trainAdaboost(
    Xtr, ytr, nMod, learning_rate):
models= np.arange(nMod, dtype=
    DecisionTreeClassifier)
N=Xtr.shape[0]
w=np.ones((N))*(1/N)
alphas=np.zeros((nMod))
for i in range(nMod):
    mdl=DecisionTreeClassifier(
        criterion='gini',max_depth=1)
    mdl.fit(Xtr,ytr,sample_weight=w)
    pred=mdl.predict(Xtr)
    models[i]=mdl
    err=sum(w*np.heaviside(-ytr*pred,1))
    if err < 1e-10: err = 1e-10
    elif err >= 1.0 - 1e-10:err = 1.0 - 1e-10
    alpha_classic=0.5*np.log((1-err)/err)
    alpha=learning_rate*alpha_classic
    models[i].alpha=alpha
    v=np.zeros((N))
    for j in range(N):
        v[j]=w[j]*np.exp(
            -alpha_classic*ytr[j]*pred[j])
    Sm=np.sum(v)
    w=v/Sm;
return models
```

```python
pca_analysis(X, y, variance_threshold=0.95):
X, y = np.asarray(X), np.asarray(y)
pca_model = PCA()
pca_model.fit(X)
cum_var = np.cumsum(
    pca_model.explained_variance_ratio_)
n_components_95 = np.argmax(
    cum_var >= variance_threshold) + 1
evr = pca_model.explained_variance_ratio_
return evr, n_components_95, pca_model, X, y
```

```python
pca_kaiser(pca_model, x, y):
eigenvalues = pca_model.explained_variance_
evr = pca_model.explained_variance_ratio_
kai_idxs = np.where(eigenvalues > 1)[0]
num_components = len(kai_idxs)
var_ret = (np.sum(eigenvalues[kai_idxs]
    ** 2) / np.sum(eigenvalues ** 2)) * 100
pca_kai = PCA(n_components=1)
X1D = pca_kai.fit_transform(x)
return num_components, kai_idxs, evr[kai_idxs]
```

```python
def test_bayesian(model, X, y):
    X = np.asarray(X)
    y_true = np.asarray(y)
    def pdfGauss(X,mean,cov):
        covInv=np.linalg.inv(cov)
        dim=cov.shape[0]
        val=np.array([])
        for i in range(X.shape[0]):
            diff = np.array([X[i,:] - mean])
            dist = (diff @ covInv @ diff.T).squeeze()
            multivariate_pdf = np.exp(-0.5*dist)/((2*np.pi)
                        **(dim/2)*np.linalg.det(cov)**0.5)
            val=np.append(val, multivariate_pdf)
        return np.array([val]).T
    def useBayes(Xte, model):
        Pw0X = pdfGauss(Xte, model['mean0'],
                        model['cov0']) * model['Pw0']
        Pw1X = pdfGauss(Xte, model['mean1'],
                        model['cov1']) * model['Pw1']
        decision_scores = (Pw1X - Pw0X).squeeze()
        y_pred_01 = np.where(decision_scores > 0, 1, 0)
        return y_pred_01, decision_scores
    feature_names = model["feature_names"]
    X_test = X
    y_pred, decision_scores = useBayes(X_test, model)
    return ... # relevant metrics? depends on exercise
```

```python
def test_mahalanobis_distance(model,
    data=None , ixHealthy=None, ixCancer=None):
    if ixHealthy is None or ixCancer is None:
        ixHealthy = np.where(data["Classification"] == "Healthy")
        ixCancer = np.where(data["Classification"] == "Cancer")
    feature_names = model["feature_names"]
    X_test = data[feature_names].to_numpy()
    y_true = np.ones(len(X_test))
    y_true[ixHealthy], y_true[ixCancer] = 0, 1
    muH = model["mu_healthy"]
    muC = model["mu_cancer"]
    Ci  = model["inv_cov"]
    if X_test.shape[1] == 1:
        muH = muH.flatten() if muH.ndim > 0 else muH
        muC = muC.flatten() if muC.ndim > 0 else muC
        X_test = X_test.flatten() if X_test.ndim > 1 else X_test
        inv_cov = Ci
        dx = ((X_test - 0.5*(muH + muC)) * (muC - muH)) * inv_cov
    else:
        dx = ((muC - muH).T @ Ci @ (X_test.T -
            0.5*(muC + muH)[:, np.newaxis])).flatten()
    y_pred = np.zeros_like(y_true)
    y_pred[dx > 0] = 1
    ...
    return ... # relevant metrics
```

```python
def test_adaboost_custom(model, X, y):
    X_test = np.asarray(X)
    #y_true = np.asarray(y)
    # feature_names = model["feature_names"]
    models = model["models"]
    # n_mod = model["n_mod"]
    def testAdaboost(Xte, models):
        nMod=models.shape[0]
        nsamp=Xte.shape[0]
        predTot=np.zeros((nMod,nsamp))
        for m in range(nMod):
            label=models[m].predict(Xte)
            predTot[m,:]=models[m].alpha*label
        decision_scores = np.sum(predTot,0)
        y_pred_pm1 = np.sign(decision_scores)
        return y_pred_pm1, decision_scores
    y_pred_pm1, decision_scores = testAdaboost(
                                X_test, models)
    ...
    #return metrics
```